1 Roteiro Prático 05 (Parte 2) — Tarefa Guiada: Refatoração da Vitrine de Serviços (Flask & SQLite)

Objetivos de Aprendizagem — Roteiro 05 (Parte 2: Tarefa Guiada)

Objetivos da Tarefa Prática (Refatoração Arquitetural da Vitrine de Serviços)
Ao longo desta tarefa de implementação orientada passo a passo, o estudante irá:

- Desenvolver e estruturar o backend RESTful em **Python Flask** com suporte a CORS para fornecer endpoints de consulta (`GET /api/servicos`) e cadastro (`POST /api/servicos`);
- Configurar a persistência relacional com **SQLite 3** em arquivo físico (`vitrine.db`), aplicando comandos SQL de criação de tabelas e inserção parametrizada para prevenir injeção de SQL;
- Refatorar o módulo de serviços do frontend (`vitrineService.js`) com a moderna **Fetch API** (`async/await`), substituindo o acesso síncrono ao `localStorage`;
- Implementar o padrão profissional de **Contingência Resiliente (Fallback)**: se a API do Flask responder, a aplicação consome o banco de dados; caso o servidor caia ou a página rode no GitHub Pages, os dados são lidos/gravados no `localStorage` em modo de cache de segurança;
- Enriquecer a camada de visualização (`vitrineView.js`) com **Spinners de Carregamento** reativos e **Badges dinâmicos de conectividade** com a API (*"Conectado à API (SQLite)"* vs *"Modo Offline (LocalStorage)"*);
- Adaptar o controlador orquestrador (`main.js`) para lidar com promessas assíncronas no carregamento e na submissão do formulário modal;
- Executar baterias de testes práticos de integração e validar a resiliência contra quedas propositais do servidor.

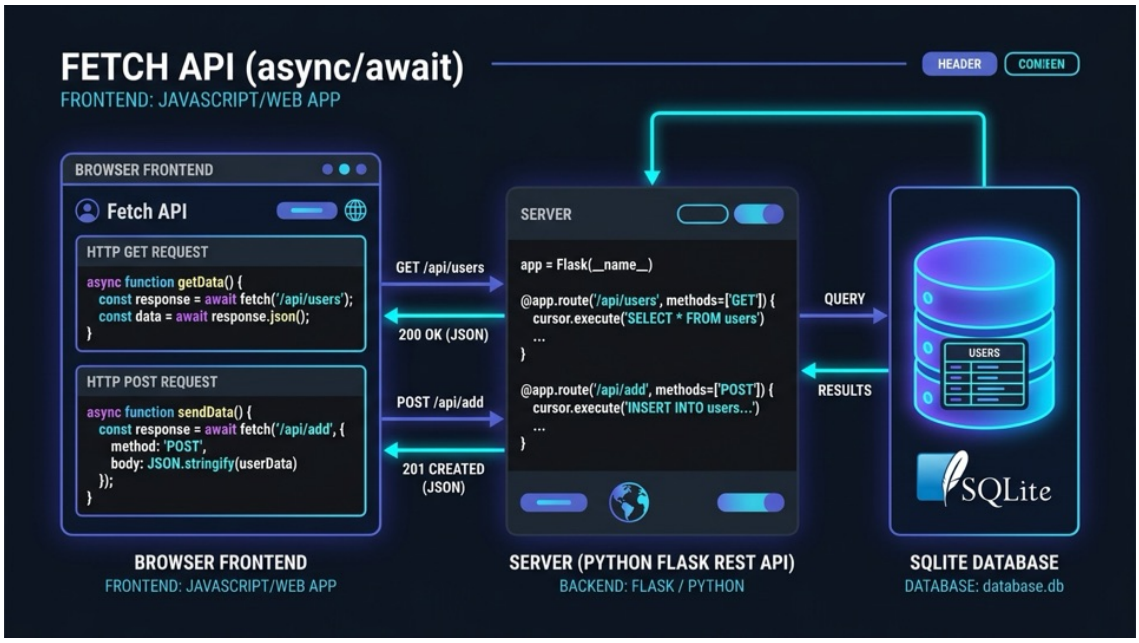


Figura 1: Visão Geral da Aplicação Integrada: Vitrine de Serviços conectada via Fetch API com backend Flask, SQLite persistente e contingência offline.

1.1 Estrutura do Projeto Integrado (`exercicio-roteiro5/`)

Para esta tarefa de refatoração, o projeto está estruturado de forma desacoplada entre cliente (*frontend*) e servidor (*backend*):

Terminal / Shell

```
roteiro-5/
|-- exercicio-roteiro5/
    |-- backend/
    |   |-- app.py                # Servidor Flask com rotas REST e SQLite
    |   |-- requirements.txt      # Dependências (Flask, Flask-CORS)
    |   \-- vitrine.db            # Banco SQLite (gerado automaticamente)
    |-- css/
    |   \-- styles.css            # Estilos customizados e animações
    |-- js/
    |   |-- services/
    |   |   \-- vitrineService.js # Camada de dados assíncrona (Fetch + Fallback)
    |   |-- views/
    |   |   \-- vitrineView.js    # Renderização DOM (cards, spinners, badges)
    |   |-- utils/
    |   |   \-- formatters.js     # Funções utilitárias puras
    |   \-- main.js               # Orquestrador assíncrono e eventos
    \-- index.html                # Interface HTML5 + Bootstrap 5 + Modal
```

1.2 Implementação Passo a Passo Guiada

Seguiremos a sequência didática em 6 fases de implementação estruturadas. Todo o código fornecido é **100% completo, sem reticências ou omissões**.

Fase 1/6: Configuração do Backend em Flask & SQLite (backend/)

Responsabilidade Arquitetural: Criar o ambiente isolado Python (virtualenv), instalar o microframework Flask com suporte a CORS e implementar a API RESTful com conexão ao SQLite em arquivo (`vitrine.db`).

Passo 1.1: Criar o arquivo de dependências (`backend/requirements.txt`)

Terminal / Shell

```
Flask>=3.0.0
flask-cors>=4.0.0
```

Passo 1.2: No terminal do VS Code, criar o ambiente virtual e instalar dependências:

Terminal / Shell

```
# Acesse a pasta do backend
cd roteiro-5/exercicio-roteiro5/backend

# Cria o ambiente virtual isolado
python3 -m venv venv

# Ativa o ambiente virtual (Linux/Mac)
source venv/bin/activate
# No Windows PowerShell utilize: .\venv\Scripts\Activate.ps1

# Instala o Flask e o CORS
pip install -r requirements.txt
```

Passo 1.3: Criar o arquivo completo do servidor (`backend/app.py`):

```
import os
import sqlite3
from datetime import datetime
from flask import Flask, jsonify, request
from flask_cors import CORS

app = Flask(__name__)
# Habilita CORS para permitir que o Live Server (porta 5500) acesse o Flask (5000)
CORS(app)

DB_PATH = os.path.join(os.path.dirname(__file__), 'vitrine.db')

def obter_conexao():
    """Conecta ao SQLite retornando linhas como dicionários."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn
```

```
def inicializar_banco():
    """Cria a tabela servicos e popula com os 3 registros modelo de Garopaba se vazia."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS servicos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            categoria TEXT NOT NULL,
            bairro TEXT NOT NULL,
            precoBase REAL NOT NULL,
            telefone TEXT NOT NULL,
            descricao TEXT,
            dataCadastro TEXT
        )
    ''')

    cursor.execute('SELECT COUNT(*) AS total FROM servicos')
    if cursor.fetchone()['total'] == 0:
        dados_semente = [
            (
                'Maré Alta Artesanatos & Cerâmicas', 'Artesanato', 'Centro Histórico',
                35.00, '48991234567',
                'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.
    ',
                datetime.now().isoformat()
            ),
            (
                'Garopaba Web & Design Studio', 'Tecnologia', 'Ferrugem',
                150.00, '48998765432',
                'Criação de websites profissionais responsivos, cardápios digitais e suporte para
    comércio local.',
                datetime.now().isoformat()
            ),
            (
                'Pescado Fresco do Zequinha', 'Alimentação', 'Canto das Canoas',
                42.00, '48984561234',
                'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e
    pontualidade.',
                datetime.now().isoformat()
            )
        ]
        cursor.executemany('''
            INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro
    )
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', dados_semente)
        conn.commit()
    conn.close()

@app.route('/api/status', methods=['GET'])
def verificar_status():
    """Diagnóstico de conectividade com a API."""
    return jsonify({
        'status': 'online',
        'mensagem': 'API REST da Vitrine de Garopaba operacional.',
        'banco': 'SQLite 3'
    }), 200

@app.route('/api/servicos', methods=['GET'])
def listar_servicos():
```

```
    """READ: Retorna a lista de todos os serviços ordenados pelo ID decrescente."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM servicos ORDER BY id DESC')
    linhas = cursor.fetchall()
    conn.close()
    return jsonify([dict(l) for l in linhas]), 200

@app.route('/api/servicos', methods=['POST'])
def cadastrar_servico():
    """CREATE: Grava um novo serviço enviado em formato JSON no SQLite."""
    dados = request.get_json()
    if not dados:
        return jsonify({'erro': 'Payload inválido.'}), 400

    conn = obter_conexao()
    cursor = conn.cursor()
    data_atual = datetime.now().isoformat()

    cursor.execute('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (
        dados['nome'].strip(),
        dados['categoria'].strip(),
        dados['bairro'].strip(),
        float(dados['precoBase']),
        dados['telefone'].strip(),
        dados.get('descricao', '').strip(),
        data_atual
    ))
    conn.commit()
    novo_id = cursor.lastrowid
    conn.close()

    return jsonify({
        'id': novo_id,
        'nome': dados['nome'].strip(),
        'categoria': dados['categoria'].strip(),
        'bairro': dados['bairro'].strip(),
        'precoBase': float(dados['precoBase']),
        'telefone': dados['telefone'].strip(),
        'descricao': dados.get('descricao', '').strip(),
        'dataCadastro': data_atual
    }), 201

@app.route('/api/servicos/<int:id>', methods=['DELETE'])
def excluir_servico(id):
    """DELETE: Exclui um serviço pelo ID."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('DELETE FROM servicos WHERE id = ?', (id,))
    conn.commit()
    conn.close()
    return jsonify({'mensagem': f'Serviço {id} removido.'}), 200

if __name__ == '__main__':
    inicializar_banco()
    print('Servidor Flask ativo na porta 5000: http://127.0.0.1:5000')
    app.run(host='127.0.0.1', port=5000, debug=True)
```

Fase 2/6: Executando e Testando a API RESTful

Responsabilidade Arquitetural: Iniciar o servidor Flask na porta 5000 e validar os endpoints de saúde (`/api/status`) e listagem (`/api/servicos`) diretamente no terminal e navegador antes da integração.

Passo 2.1: Iniciar o servidor no terminal:

```
Terminal / Shell
python app.py
```

Passo 2.2: Testar no navegador ou via cURL em outra aba do terminal:

```
Terminal / Shell
curl http://127.0.0.1:5000/api/status
```

Resposta esperada do servidor (JSON com status 200):

```
Terminal / Shell
{"banco":"SQLite 3","mensagem":"API REST da Vitrine de Garopaba operacional.","status":"online"}
```

Checkpoint de Verificação

Checkpoint de Validação da Fase 2 Abra no navegador `http://127.0.0.1:5000/api/servicos`. Você deverá ver a lista JSON com os 3 registros semente cadastrados automaticamente no arquivo `vitrine.db`.

Fase 3/6: A Revolução Assíncrona no vitrineService.js

Responsabilidade Arquitetural: Refatorar os métodos de leitura e escrita para utilizarem requisições HTTP assíncronas com `fetch()` e `async/await`, mantendo uma camada de contingência transparente com `localStorage`.

```
// js/services/vitrineService.js
const API_BASE_URL = 'http://127.0.0.1:5000/api';
const STORAGE_KEY = 'garopaba_vitrine_servicos';

let apiConectada = false;

export const DADOS_INICIAIS = [
  {
    id: '1',
    nome: 'Maré Alta Artesanatos & Cerâmicas',
    categoria: 'Artesanato',
    bairro: 'Centro Histórico',
    precoBase: 35.00,
    telefone: '48991234567',
    descricao: 'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.'
  },
```

```
  {
    id: '2',
    nome: 'Garopaba Web & Design Studio',
    categoria: 'Tecnologia',
    bairro: 'Ferrugem',
    precoBase: 150.00,
    telefone: '48998765432',
    descricao: 'Criação de websites profissionais responsivos, cardápios digitais e suporte para
     comércio local.'
  },
  {
    id: '3',
    nome: 'Pescado Fresco do Zequinha',
    categoria: 'Alimentação',
    bairro: 'Canto das Canoas',
    precoBase: 42.00,
    telefone: '48984561234',
    descricao: 'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e
     pontualidade.'
  }
];

export function estaConectadoNaApi() {
  return apiConectada;
}

export async function testarConexaoApi() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/status`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(2000)
    });
    apiConectada = resposta.ok;
    return apiConectada;
  } catch (erro) {
    apiConectada = false;
    return false;
  }
}

// Rotinas Locais de Contingência (Fallback)
function obterServicosLocal() {
  const dados = localStorage.getItem(STORAGE_KEY);
  if (!dados) {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(DADOS_INICIAIS));
    return DADOS_INICIAIS;
  }
  try { return JSON.parse(dados); } catch (e) { return []; }
}

function salvarServicoLocal(novoServico) {
  const servicos = obterServicosLocal();
  const servicoCompleto = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novoServico
  };
  servicos.unshift(servicoCompleto);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicoCompleto;
}
```

```
function removerServicoLocal(id) {
  const servicos = obterServicosLocal();
  const filtrados = servicos.filter(s => s.id.toString() !== id.toString());
  localStorage.setItem(STORAGE_KEY, JSON.stringify(filtrados));
  return filtrados;
}

// Operações Assíncronas com Fetch API
export async function obterServicos() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(3000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const dados = await resposta.json();
    apiConectada = true;
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dados));
    return dados;
  } catch (erro) {
    apiConectada = false;
    console.warn('API Flask offline. Ativando contingência LocalStorage:', erro.message);
    return obterServicosLocal();
  }
}

export async function salvarServico(novoServico) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json'
      },
      body: JSON.stringify(novoServico),
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const criado = await resposta.json();
    apiConectada = true;
    const servicos = obterServicosLocal();
    servicos.unshift(criado);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return criado;
  } catch (erro) {
    apiConectada = false;
    console.warn('Falha no envio à API. Salvando localmente:', erro.message);
    return salvarServicoLocal(novoServico);
  }
}

export async function removerServico(id) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos/${id}`, {
      method: 'DELETE',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    apiConectada = true;
```

```
    removerServicoLocal(id);
    return true;
  } catch (erro) {
    apiConectada = false;
    console.warn('Falha na exclusão remota. Removendo localmente:', erro.message);
    removerServicoLocal(id);
    return true;
  }
}
```

Fase 4/6: Camada de Visão & Feedbacks de Rede (vitrineView.js)

Responsabilidade Arquitetural: Adicionar componentes visuais de carregamento (`exibirSpinner`) e indicador de conectividade em tempo real com a API no topo da página.

```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml } from '../utils/formatters.js';

const CORES_CATEGORIA = {
  'Alimentação': 'success',
  'Tecnologia': 'info',
  'Artesanato': 'warning',
  'Serviços Gerais': 'primary',
  'Turismo': 'secondary'
};

export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');

  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
              <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>

          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>

          <hr class="my-3 text-muted opacity-25">

          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank" class="btn btn-sm btn-outline-
    success rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2" data-id="${s.
    id}" title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
```

```
      </div>
    </div>
  `;
}

export function renderizarCards(servicos, containerElement) {
  if (!containerElement) return;
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 py-5 text-center text-muted">
        <i class="bi bi-inbox fs-1 d-block mb-3 text-secondary opacity-50"></i>
        <h5 class="fw-bold">Nenhum serviço encontrado</h5>
        <p class="small">Cadastre um novo serviço ou altere o filtro de categoria.</p>
      </div>
    `;
    return;
  }
  containerElement.innerHTML = servicos.map(s => criarCardHtml(s)).join('');
}

export function exibirSpinner(containerElement) {
  if (!containerElement) return;
  containerElement.innerHTML = `
    <div class="col-12 py-5 text-center text-primary">
      <div class="spinner-border text-primary" role="status" style="width: 3rem; height: 3rem;">
        <span class="visually-hidden">Carregando dados da API...</span>
      </div>
      <p class="mt-3 text-muted small fw-semibold">Consultando serviços no backend Flask (SQLite)...</p
    >
    </div>
  `;
}

export function atualizarStatusConexao(isOnline, statusBadgeEl) {
  if (!statusBadgeEl) return;
  if (isOnline) {
    statusBadgeEl.className = 'badge bg-success-subtle text-success border border-success-subtle px-3
     py-2 rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-cloud-check-fill me-1"></i> Backend Online (Flask +
     SQLite)';
  } else {
    statusBadgeEl.className = 'badge bg-warning-subtle text-warning border border-warning-subtle px-3
     py-2 rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-hdd-network-fill me-1"></i> Modo Offline (LocalStorage)'
     ;
  }
}

export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastMsgEl = document.getElementById('toastMensagem');
  if (!toastEl || !toastMsgEl) return;
  toastEl.className = `toast align-items-center text-white bg-${tipo} border-0 shadow-lg`;
  toastMsgEl.innerText = mensagem;
  const bsToast = bootstrap.Toast.getOrCreateInstance(toastEl, { delay: 3500 });
  bsToast.show();
}
```

Fase 5/6: Orquestrador Assíncrono da Aplicação (main.js)

Responsabilidade Arquitetural: Coordenar os fluxos de eventos da interface utilizando `await` para aguardar a persistência no banco remoto antes de fechar modais e atualizar a vitrine.

```
// js/main.js
import {
  obterServicos,
  salvarServico,
  removerServico,
  testarConexaoApi,
  estaConectadoNaApi
} from './services/vitrineService.js';

import {
  renderizarCards,
  exibirSpinner,
  atualizarStatusConexao,
  exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', async () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const statusApiBadge = document.getElementById('statusApiBadge');
  const modalEl = document.getElementById('modalCadastro');
  const btnSalvarTexto = document.getElementById('btnSalvarTexto');
  const btnSalvar = document.getElementById('btnSalvar');

  let cacheServicosAtuais = [];

  async function atualizarInterface() {
    exibirSpinner(container);
    const conectado = await testarConexaoApi();
    atualizarStatusConexao(conectado, statusApiBadge);

    cacheServicosAtuais = await obterServicos();
    const categoriaSelecionada = filtroCategoria ? filtroCategoria.value : 'todas';
    const servicosFiltrados = categoriaSelecionada === 'todas'
      ? cacheServicosAtuais
      : cacheServicosAtuais.filter(s => s.categoria === categoriaSelecionada);

    renderizarCards(servicosFiltrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${cacheServicosAtuais.length} Serviços Cadastrados`;
    }
  }

  if (filtroCategoria) {
    filtroCategoria.addEventListener('change', () => {
      const categoria = filtroCategoria.value;
      const filtrados = categoria === 'todas'
        ? cacheServicosAtuais
        : cacheServicosAtuais.filter(s => s.categoria === categoria);
      renderizarCards(filtrados, container);
    });
```

```
}

// Inicialização assíncrona
await atualizarInterface();

// Reset do formulário ao abrir modal
if (modalEl) {
  modalEl.addEventListener('show.bs.modal', () => {
    form.reset();
    form.classList.remove('was-validated');
  });
}

function obterValorCampo(idCampo) {
  const elemento = document.getElementById(idCampo);
  if (elemento) {
    return elemento.value.trim();
  }
  return '';
}

// Cadastro de Novo Serviço via HTTP POST (Função Nomeada)
async function cadastrarNovoServico(evento) {
  evento.preventDefault();
  if (form.checkValidity() === false) {
    evento.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

  const dados = {
    nome: obterValorCampo('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: obterValorCampo('bairro'),
    precoBase: parseFloat(obterValorCampo('precoBase')) || 0,
    telefone: obterValorCampo('telefone'),
    descricao: obterValorCampo('descricao')
  };

  if (btnSalvar) btnSalvar.disabled = true;
  if (btnSalvarTexto) btnSalvarTexto.innerText = 'Gravando...';

  try {
    await salvarServico(dados);
    let msg = 'Serviço salvo localmente (Modo Offline).';
    if (estaConectadoNaApi() === true) {
      msg = 'Serviço gravado no banco de dados SQLite!';
    }
    exibirToast(msg, 'success');

    const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
    modal.hide();
    await atualizarInterface();
  } catch (erro) {
    exibirToast('Erro ao processar cadastro.', 'danger');
  } finally {
    if (btnSalvar) btnSalvar.disabled = false;
    if (btnSalvarTexto) btnSalvarTexto.innerText = 'Salvar no Banco';
  }
}

form.addEventListener('submit', cadastrarNovoServico);
```

```
  // Exclusão Assíncrona via HTTP DELETE (Função Nomeada)
  async function tratarExclusaoNaVitrine(evento) {
    const btnExcluir = evento.target.closest('.btn-excluir');
    if (btnExcluir) {
      const id = btnExcluir.getAttribute('data-id');
      if (confirm('Deseja realmente remover este serviço da vitrine?')) {
        await removerServico(id);
        await atualizarInterface();
        exibirToast('Serviço excluído da vitrine.', 'warning');
      }
    }
  }

  container.addEventListener('click', tratarExclusaoNaVitrine);
});
```

Fase 6/6: Verificação de Integração & Prova de Contingência

Responsabilidade Arquitetural: Executar o roteiro de testes em dois cenários: com a API online no laboratório e com o servidor desligado comprovando a resiliência offline.

1. Teste Conectado (API Online):

- Inicie o backend com `python app.py` na porta `5000`;
- Abra o frontend via Live Server (`http://127.0.0.1:5500/index.html`);
- O badge no topo deve exibir: ***Backend Online (Flask + SQLite)***;
- Cadastre um novo serviço e observe a requisição HTTP POST sendo logada no terminal do Flask!

2. Teste de Resiliência (Fallback LocalStorage):

- No terminal do Flask, pressione `Ctrl + C` para desligar o servidor;
- Recarregue a página no navegador com `F5`;
- O frontend detecta a indisponibilidade e muda o badge para: ***Modo Offline (LocalStorage)***;
- Os cards continuam aparecendo normalmente a partir da cópia em cache!

Checkpoint de Verificação

Critério de Conclusão da Tarefa Principal A aplicação demonstrou sucesso na comunicação assíncrona cliente-servidor via Fetch API, realizou persistência no SQLite e provou resiliência arquitetural ao operar em modo de contingência offline.

1.3 Mini Desafios Práticos de Extensão & Fixação

Agora que sua arquitetura full-stack está completamente funcional e resiliente, propomos **3 mini desafios progressivos** para você consolidar como novas funcionalidades trafegam por todas as camadas do sistema (Banco SQLite → Backend Flask → Frontend JavaScript → Interface HTML5/Bootstrap).

Metodologia de Aprendizagem Ativa

Recomendamos fortemente que você tente implementar cada um dos 3 desafios por conta própria no código do projeto antes de consultar o gabarito! Observe quais arquivos precisam ser ajustados e teste os resultados no navegador.

Mini Desafio 1: Adicionar o Campo Booleano "Aceita PIX"

Contexto: Os comerciantes locais solicitaram que a vitrine indique claramente se o estabelecimento aceita pagamentos instantâneos via PIX.

- **No Banco SQLite e Backend (`backend/app.py`):** Adicionar a coluna `aceitaPix INTEGER DEFAULT 1` na tabela `servicos`. Na rota `POST /api/servicos`, capturar o booleano enviado no JSON e salvá-lo como `1` ou `0`;
- **No Formulário Modal (`index.html`):** Inserir um interruptor (*switch*) do Bootstrap com o identificador `id="aceitaPix"`, marcado como ativo por padrão (`checked`);
- **No Orquestrador (`js/main.js`):** Coletar o valor booleano com `document.getElementById('aceitaPix')` e incluí-lo no objeto enviado ao backend;
- **Na Camada de Visão (`js/views/vitrineView.js`):** Se `s.aceitaPix` for verdadeiro, renderizar um badge verde elegante ao lado do badge de categoria no topo do card.

Mini Desafio 2: Contador Reativo de Serviços Cadastrados

Contexto: A interface atual lista os serviços, mas os usuários não conseguem visualizar imediatamente quantos estabelecimentos estão cadastrados no total.

- **Na Interface (`index.html`):** Adicionar um badge de contador ao lado do subtítulo ou na barra de filtros com `id="totalServicosBadge"`;
- **Na Camada de Visão (`js/views/vitrineView.js`):** Criar a função exportada `atualizarContadorServico` `elemento)` que exiba o texto no singular (*"1 serviço cadastrado"*) ou no plural (*"N serviços cadastrados"*);
- **No Orquestrador (`js/main.js`):** Invocar a atualização do contador sempre que a lista for carregada da API e também imediatamente após a exclusão de um registro.

Mini Desafio 3: Campo de Seleção "Prazo de Atendimento"

Contexto: Diferentes serviços possuem prazos distintos (ex: pesca artesanal com entrega imediata vs estúdio de design sob agendamento prévio).

- **No Banco e Backend (`backend/app.py`):** Adicionar a coluna `prazoAtendimento TEXT DEFAULT 'Sob agendamento'` na tabela `servicos` e persistir o valor recebido no `POST`;
- **No Modal (`index.html`):** Adicionar um campo `select` com `id="prazoAtendimento"` oferecendo opções como *"Atendimento Imediato"*, *"Em até 24 horas"*, *"2 a 5 dias úteis"* e *"Sob agendamento"*;
- **No Orquestrador (`js/main.js`):** Coletar o valor selecionado no formulário e adicioná-lo ao payload do cadastro;
- **Na Camada de Visão (`js/views/vitrineView.js`):** Exibir uma linha no corpo do card com o ícone de relógio informando o prazo de atendimento do profissional.

1.4 Gabarito Comentado dos Mini Desafios (Código Completo)

Confira a seguir as soluções completas de cada desafio, organizadas por arquivo. Todas as instruções seguem o princípio de código 100% aplicável (*Zero-Ellipsis Rule*).

Solução do Mini Desafio 1: Campo Booleano "Aceita PIX"

Passo 1.1: No arquivo `backend/app.py`, ajuste a criação da tabela e o endpoint de cadastro:

```
# No método inicializar_banco(), adicione a coluna aceitaPix:
cursor.execute('''
    CREATE TABLE IF NOT EXISTS servicos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        categoria TEXT NOT NULL,
        bairro TEXT NOT NULL,
        precoBase REAL NOT NULL,
        telefone TEXT NOT NULL,
        descricao TEXT,
        dataCadastro TEXT,
        aceitaPix INTEGER DEFAULT 1
    )
''')

# No endpoint cadastrar_servico(), colete o valor booleano e converta para 1 ou 0:
@app.route('/api/servicos', methods=['POST'])
def cadastrar_servico():
    dados = request.get_json()
    if not dados:
        return jsonify({'erro': 'Payload inválido.'}), 400

    conn = obter_conexao()
    cursor = conn.cursor()
    data_atual = datetime.now().isoformat()
    aceita_pix = 1 if dados.get('aceitaPix', True) else 0

    cursor.execute('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro,
     aceitaPix)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    ''', (
        dados['nome'].strip(),
        dados['categoria'].strip(),
        dados['bairro'].strip(),
        float(dados['precoBase']),
        dados['telefone'].strip(),
        dados.get('descricao', '').strip(),
        data_atual,
        aceita_pix
    ))
    conn.commit()
    novo_id = cursor.lastrowid
    conn.close()

    return jsonify({
        'id': novo_id,
        'mensagem': 'Serviço cadastrado com sucesso!',
        'aceitaPix': bool(aceita_pix)
```

```
    }), 201
```

Passo 1.2: No arquivo `index.html`, adicione o Switch no Modal de Cadastro

Insira o controle logo abaixo do campo de Telefone (WhatsApp):

```
<div class="mb-3">
  <div class="form-check form-switch p-2 ps-5 bg-light rounded-3 border">
    <input class="form-check-input" type="checkbox" role="switch" id="aceitaPix" checked>
    <label class="form-check-label fw-semibold text-dark" for="aceitaPix">
      <i class="bi bi-qr-code text-success me-1"></i> Aceita pagamento instantâneo via PIX
    </label>
  </div>
</div>
```

Passo 1.3: No arquivo `js/main.js`, colete o valor do checkbox

Na função de submissão do formulário (`cadastrarNovoServico`), adicione a propriedade `aceitaPix`:

```
const dados = {
  nome: obterValorCampo('nome'),
  categoria: document.getElementById('categoria').value,
  bairro: obterValorCampo('bairro'),
  precoBase: parseFloat(obterValorCampo('precoBase')) || 0,
  telefone: obterValorCampo('telefone'),
  descricao: obterValorCampo('descricao'),
  aceitaPix: document.getElementById('aceitaPix').checked
};
```

Passo 1.4: No arquivo `js/views/vitrineView.js`, renderize o Badge no Card

Atualize a função `criarCardHtml(s)` para renderizar condicionalmente o badge do PIX:

```
export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');
  const badgePixHtml = (s.aceitaPix === 1 || s.aceitaPix === true)
    ? '<span class="badge bg-success-subtle text-success border border-success-subtle rounded-pill font
    -outfit px-2 py-1"><i class="bi bi-qr-code me-1"></i>PIX</span>'
    : '';

  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
```

```
            <div class="d-flex align-items-center gap-1">
              <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
                <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
              </span>
              ${badgePixHtml}
            </div>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>

          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>

          <hr class="my-3 text-muted opacity-25">

          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank" class="btn btn-sm btn-outline-
    success rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2" data-id="${s.
    id}" title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  `;
}
```

Solução do Mini Desafio 2: Contador Reativo de Serviços

Passo 2.1: No arquivo `index.html`, adicione o elemento badge no cabeçalho da página:

```
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4">
  <div>
    <h1 class="h3 mb-1 text-primary fw-bold">
      <i class="bi bi-shop me-2"></i>Vitrine de Serviços de Garopaba
    </h1>
    <p class="text-muted small mb-0">
      Mural comunitário de prestadores locais e comércio da região.
    </p>
  </div>
  <div class="d-flex align-items-center gap-2">
    <span id="totalServicosBadge" class="badge bg-primary-subtle text-primary border border-primary-
    subtle rounded-pill px-3 py-2 fw-semibold">
      <i class="bi bi-collection me-1"></i> Carregando...
    </span>
    <span id="statusConexao" class="badge bg-secondary-subtle text-secondary border px-3 py-2 rounded-
    pill fw-semibold">
      <i class="bi bi-arrow-repeat me-1"></i> Verificando conexão...
    </span>
  </div>
</div>
```

Passo 2.2: No arquivo `js/views/vitrineView.js`, crie a função de contagem

Exporte a nova função utilitária de renderização de contadores:

```
export function atualizarContadorServicos(total, elementoBadge) {
  if (!elementoBadge) return;
  const texto = total === 1 ? '1 serviço ativo' : `${total} serviços ativos`;
  elementoBadge.innerHTML = `<i class="bi bi-collection me-1"></i> ${texto}`;
}
```

Passo 2.3: No arquivo `js/main.js`, conecte a contagem ao fluxo

Importe `atualizarContadorServicos` e atualize nas rotinas de carregamento e filtragem:

```
// 1. Atualize o cabeçalho de importações de ./views/vitrineView.js:
import {
  renderizarCards,
  exibirSpinner,
  atualizarStatusConexao,
  atualizarContadorServicos,
  exibirToast
} from './views/vitrineView.js';

// 2. Atualize a função de renderização da vitrine:
```

```
async function atualizarInterface() {
  exibirSpinner(container);
  const badgeTotal = document.getElementById('totalServicosBadge');

  try {
    const todosServicos = await obterServicos();
    const categoriaSelecionada = selectFiltro ? selectFiltro.value : 'todas';

    let servicosFiltrados = todosServicos;
    if (categoriaSelecionada !== 'todas') {
      servicosFiltrados = todosServicos.filter(s => s.categoria === categoriaSelecionada);
    }

    renderizarCards(servicosFiltrados, container);
    atualizarContadorServicos(servicosFiltrados.length, badgeTotal);
  } catch (erro) {
    exibirToast('Erro ao atualizar a vitrine de serviços.', 'danger');
  }
}
```

Solução do Mini Desafio 3: Campo de Seleção "Prazo de Atendimento"

Passo 3.1: No arquivo `backend/app.py`, inclua a coluna `prazoAtendimento`:

```
# Na criação da tabela em inicializar_banco():
cursor.execute('''
    CREATE TABLE IF NOT EXISTS servicos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        categoria TEXT NOT NULL,
        bairro TEXT NOT NULL,
        precoBase REAL NOT NULL,
        telefone TEXT NOT NULL,
        descricao TEXT,
        dataCadastro TEXT,
        aceitaPix INTEGER DEFAULT 1,
        prazoAtendimento TEXT DEFAULT 'Sob agendamento'
    )
''')

# No endpoint cadastrar_servico(), colete prazoAtendimento com fallback seguro:
@app.route('/api/servicos', methods=['POST'])
def cadastrar_servico():
    dados = request.get_json()
    if not dados:
        return jsonify({'erro': 'Payload inválido.'}), 400

    conn = obter_conexao()
    cursor = conn.cursor()
    data_atual = datetime.now().isoformat()
    aceita_pix = 1 if dados.get('aceitaPix', True) else 0
    prazo = dados.get('prazoAtendimento', 'Sob agendamento').strip()

    cursor.execute('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro,
     aceitaPix, prazoAtendimento)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    ''', (
        dados['nome'].strip(),
        dados['categoria'].strip(),
        dados['bairro'].strip(),
        float(dados['precoBase']),
        dados['telefone'].strip(),
        dados.get('descricao', '').strip(),
        data_atual,
        aceita_pix,
        prazo
    ))
    conn.commit()
    novo_id = cursor.lastrowid
    conn.close()

    return jsonify({'id': novo_id, 'mensagem': 'Serviço salvo!'}), 201
```

Passo 3.2: No arquivo `index.html`, adicione o Dropdown no Modal

Insira o campo de seleção de prazo no formulário de cadastro:

```
<div class="mb-3">
  <label for="prazoAtendimento" class="form-label fw-semibold text-secondary">
    <i class="bi bi-clock-history me-1 text-primary"></i> Prazo Estimado de Atendimento
  </label>
  <select class="form-select" id="prazoAtendimento" required>
    <option value="Atendimento Imediato">Atendimento Imediato</option>
    <option value="Em até 24 horas">Em até 24 horas</option>
    <option value="2 a 5 dias úteis">2 a 5 dias úteis</option>
    <option value="Sob agendamento" selected>Sob agendamento</option>
  </select>
</div>
```

Passo 3.3: No arquivo `js/main.js`, colete o valor do campo

No manipulador de cadastro, acrescente a leitura do campo:

```
const dados = {
  nome: obterValorCampo('nome'),
  categoria: document.getElementById('categoria').value,
  bairro: obterValorCampo('bairro'),
  precoBase: parseFloat(obterValorCampo('precoBase')) || 0,
  telefone: obterValorCampo('telefone'),
  descricao: obterValorCampo('descricao'),
  aceitaPix: document.getElementById('aceitaPix').checked,
  prazoAtendimento: document.getElementById('prazoAtendimento').value
};
```

Passo 3.4: No arquivo `js/views/vitrineView.js`, exiba o Prazo no Card

Atualize `criarCardHtml(s)` para exibir a linha de prazo de atendimento logo abaixo do bairro:

```
export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');
  const badgePixHtml = (s.aceitaPix === 1 || s.aceitaPix === true)
    ? '<span class="badge bg-success-subtle text-success border border-success-subtle rounded-pill font
    -outfit px-2 py-1"><i class="bi bi-qr-code me-1"></i>PIX</span>'
    : '';
  const prazoTexto = escapeHtml(s.prazoAtendimento || 'Sob agendamento');

  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
```

```
            <div class="d-flex align-items-center gap-1">
              <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
                <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
              </span>
              ${badgePixHtml}
            </div>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>

          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-2">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <div class="text-secondary small mb-2">
            <i class="bi bi-clock-history text-primary me-1"></i><strong>Prazo:</strong> ${prazoTexto}
          </div>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>

          <hr class="my-3 text-muted opacity-25">

          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank" class="btn btn-sm btn-outline-
    success rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2" data-id="${s.
    id}" title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  `;
}
```

1.5 Quadro Comparativo: Evolução do Roteiro 04 para o Roteiro 05

Componente	Roteiro 04 (Módulo 2)	Roteiro 05 (Módulo 3)
Ambiente de Dados	LocalStorage (isolado na aba do navegador).	Banco relacional SQLite (`vitrine.db`) no Flask.
Comunicação	Chamadas de função síncronas em memória.	Chamadas assíncronas de rede com `fetch()` e `await`.
Segurança de Rede	Inexistente (tudo no cliente).	Configuração de cabeçalhos CORS (`flask-cors`).
Experiência (UX)	Resposta imediata sem feedback de rede.	Spinners de espera e badges de status de conectividade.
formatters.js	Funções puras de formatação.	**100% idêntico!** Prova de desacoplamento limpo.

Parabéns! Você implementou uma arquitetura moderna full-stack desacoplada, apta para o ecossistema profissional e pronta para avançar rumo às Single Page Applications (SPAs) e ao projeto de extensão da SNCT!